COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/Data Structures and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	July 9, 2026	**DATE SUBMITTED:**	July 11, 2026
NAME:	Gresola, Aryanna Keith G.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 1.1 Basic C++ Programming

1. PROCEDURE OUTPUT

ILO A: Create Code That Follows the Basic C++ Code Structure

1. Create a function that will take two numbers and display the sum.
2. Create a function that will return whether variable A is greater than variable B.
3. Create a function that will take two Boolean values and display the result of all logical operations then return true if it was a success.

INPUT

```cpp
#include <iostream>

//Function Prototype
void displaySum(int fNum, int sNum);
bool valGreater(int A, int B);
bool logicalOperations( bool a, bool b);


int main(){
    int num1, num2;
    int A, B;
    bool a, b;

    std::cout << "GETTING THE SUM OF TWO NUMBERS" << std::endl;
    std::cout << "Enter first number: " << std::endl;
    std::cin >> num1;
    std::cout << "Enter second number: " << std::endl;
    std::cin >> num2;
    displaySum(num1, num2);
    std::cout << std::endl;

    std::cout << "COMPARING GREATER VALUE" << std::endl;
    std::cout << "Enter value A: " << std::endl;
    std::cin >> A;
    std::cout << "Enter value B: " << std::endl;
    std::cin >> B;
    std::cout << std::endl;
```

```cpp

    if (valGreater(A,B)){
        std::cout << "A is greater than B. " << std::endl;
    }else
        std::cout << "A is NOT greater than B. " << std::endl;
    std::cout << std::endl;

    std::cout << "LOGICAL OPERATIONS" << std::endl;
    std::cout << "Enter first boolean value  (1 = True, 0 = False): " << std::endl;
    std::cin >> a;
    std::cout << "Enter second boolean value (1 = True, 0 = False): " << std::endl;
    std::cin >> b;

    if (logicalOperations(a,b))
        std::cout <<  "Logical Operations executed successfully." << std::endl;

    return 0;
}

//  Function Definition

void displaySum(int fNum, int sNum){
    std::cout << "The sum of the two number is: " << fNum + sNum << std::endl;
}
bool valGreater(int A, int B){
    return A > B;
}
bool logicalOperations(bool a, bool b){
    std::cout << "AND: " << (a && b) << std::endl;
    std::cout << "OR: " << (a || b) << std::endl;
    std::cout << "NOT a: " << (!a) << std::endl;
    std::cout << "NOT b: " << (!b) << std:: endl;
    return true;
}
```

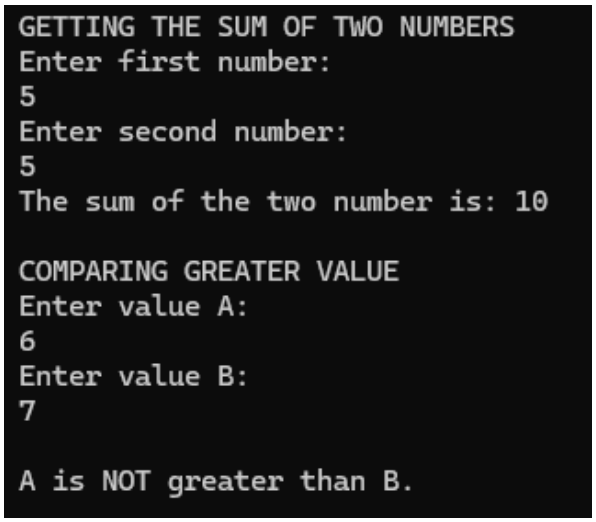
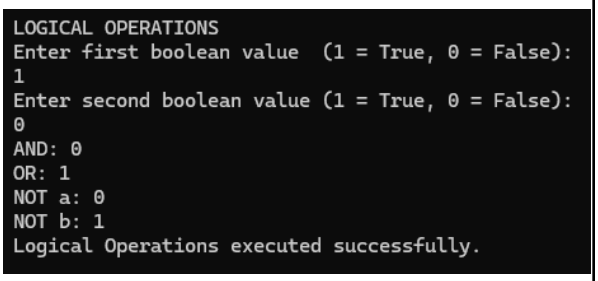
OUTPUT

```
GETTING THE SUM OF TWO NUMBERS
Enter first number:
5
Enter second number:
5
The sum of the two number is: 10

COMPARING GREATER VALUE
Enter value A:
6
Enter value B:
5

A is greater than B.
```

```
LOGICAL OPERATIONS
Enter first boolean value  (1 = True, 0 = False):
1
Enter second boolean value (1 = True, 0 = False):
1
AND: 1
OR: 1
NOT a: 0
NOT b: 0
Logical Operations executed successfully.
```

CODE ANALYSIS
This program demonstrates the use of function prototypes, user-defined functions, and Boolean operations in C++. It begins by declaring function prototypes, which inform the compiler about each function's name, return type, and parameters before their actual definitions, allowing them to be called inside the main() function. The program then accepts user input to perform three separate tasks: calculating the sum of two integers, comparing two values, and executing logical operations on Boolean inputs. Each task is handled by its own function, making the program more organized and easier to understand. The displaySum() function prints the sum of two numbers, valGreater() returns a Boolean value to determine if one number is greater than another, and logicalOperations() displays the results of the AND, OR, and NOT operators. I know this program may seem a bit hard-coded, but I intentionally structured it this way to help me further recall and reinforce the basic structure of C++, especially the use of function prototypes, function calls, and user-defined functions.

SECTIONS	ANSWER
Header File Declaration Section	`#include <iostream>`

Global Declaration Section	`//Function Prototype`
	`void displaySum(int fNum, int sNum);`
	`bool valGreater(int A, int B); S`
	`bool logicalOperations( bool a, bool b);`

Class Declaration and Method Definition Section	**N/A**

Main Function

```
int main(){
    int num1, num2;
    int A, B;
    bool a, b;

    std::cout << "GETTING THE SUM OF TWO NUMBERS" << std::endl;
    std::cout << "Enter first number: " << std::endl;
    std::cin >> num1;
    std::cout << "Enter second number: " << std::endl;
    std::cin >> num2;
    displaySum(num1, num2);
    std::cout << std::endl;

    std::cout << "COMPARING GREATER VALUE" << std::endl;
    std::cout << "Enter value A: " << std::endl;
    std::cin >> A;
    std::cout << "Enter value B: " << std::endl;
    std::cin >> B;
    std::cout << std::endl;SS

    if (valGreater(A,B)){
        std::cout << "A is greater than B. " << std::endl;
    }else
        std::cout << "A is NOT greater than B. " << std::endl;
    std::cout << std::endl;

    std::cout << "LOGICAL OPERATIONS" << std::endl;
    std::cout << "Enter first boolean value  (1 = True, 0 = False): " << std::endl;
    std::cin >> a;
    std::cout << "Enter second boolean value (1 = True, 0 = False): " << std::endl;
    std::cin >> b;

    if (logicalOperations(a,b))
        std::cout <<  "Logical Operations executed successfully." << std::endl;

    return 0;
}
```

Method Definition

```
//  Function Definition

void displaySum(int fNum, int sNum){
    std::cout << "The sum of the two number is: " << fNum + sNum << std::endl;
}
bool valGreater(int A, int B){
    return A > B;
}
bool logicalOperations(bool a, bool b){
    std::cout << "AND: " << (a && b) << std::endl;
    std::cout << "OR: " << (a || b) << std::endl;
    std::cout << "NOT a: " << (!a) << std::endl;
    std::cout << "NOT b: " << (!b) << std:: endl;
    return true;
}
```

ILO B: Implement Appropriate Class Definition and Instances Based on Given Requirements

INPUT

```cpp
#include <iostream>

// naming convention fo class name: PascalCase
class Triangle{
    private:
        double totalAngle, angleA, angleB, angleC;
    public:
        //constructor
        Triangle(double A, double B, double C);
        void setAngles(double A, double B, double C);
        const bool validateTriangle();
};


int main(){
    Triangle setl(40, 30, 110);
    if (setl.validateTriangle()){
        std::cout << "The shape is a valid triangle \n";
    }else{
        std::cout << "The shape is NOT a valid triangle \n";
    }
```

```
22
23  }
24  Triangle::Triangle(double A, double B, double C){
25      angleA = A;
26      angleB = B;
27      angleC = C;
28      totalAngle = A+B+C;
29  }
30  void Triangle::setAngles(double A, double B, double C){
31      angleA = A;
32      angleB = B;
33      angleC = C;
34      totalAngle = A+B+C;
35  }
36
37  const bool Triangle::validateTriangle(){
38      return(totalAngle <= 180);
39  }
```

OUTPUT

```
The shape is a valid triangle
```

CODE ANALYSIS

This C++ program demonstrates the use of classes, constructors, member functions, and boolean return values to determine whether three given angles can form a triangle. The Triangle class serves as a blueprint that groups the triangle's data (angleA, angleB, angleC, and totalAngle) with the functions that operate on it, demonstrating encapsulation. The constructor Triangle(double A, double B, double C) is automatically called when a Triangle object is created, initializing the angle values and calculating their total. The validateTriangle() member function returns a boolean (bool) value that is used in an if statement to determine whether the triangle is valid, while setAngles() allows the angle values to be updated later. In the main() function, a Triangle object is created with the angles 40, 30, and 110, and the result of validateTriangle() determines which message is displayed. Overall, the program shows how classes organize data and functions, constructors initialize objects, and boolean return values help make decisions in a program.

SECTIONS	ANSWERS
Header File Declaration Section	`#include <iostream>`
Global Declaration Section	**N/A:** There are no global functions declared in this program.

Class Declaration and Method Definition Section	`// naming convention fo class name: PascalCase`
	`class Triangle{`
	`  private:`
	`  double totalAngle, angleA, angleB, angleC;`
	`  public:`
	`  //constructor`
	`  Triangle(double A, double B, double C);`
	`  void setAngles(double A, double B, double C);`
	`  const bool validateTriangle();`
	`};`

Main Function	`int main(){`
	`  Triangle setl(40, 30, 110);`
	`  if (setl.validateTriangle()){`
	`  std::cout << "The shape is a valid triangle \n";`
	`  }else{`
	`  std::cout << "The shape is NOT a valid triangle \n";`
	`  }`
	
	`}`

Method Definition	`Triangle::Triangle(double A, double B, double C){`
	`  angleA = A;`
	`  angleB = B;`
	`  angleC = C;`
	`  totalAngle = A+B+C;`
	`}`
	`void Triangle::setAngles(double A, double B, double C){`
	`  angleA = A;`
	`  angleB = B;`
	`  angleC = C;`
	`  totalAngle = A+B+C;`
	`}`
	
	`const bool Triangle::validateTriangle(){`
	`  return(totalAngle <= 180);`
	`}`

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.
ILO C: Solve Different Problems using the C++ Programming Language

1. Create a C++ program to swap the two numbers in different variables.

INPUT

```
#include <iostream>

void swapNum(int &num1, int &num2); //& means pass by reference.

int main(){
    int fNum, sNum;

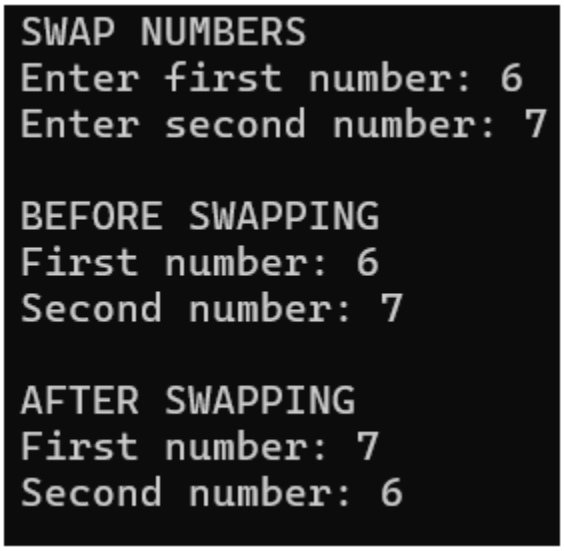
    std::cout << "SWAP NUMBERS" << std::endl;
    std::cout << "Enter first number: ";
    std::cin >> fNum;
    std::cout << "Enter second number: ";
    std::cin >> sNum;
    std::cout << std::endl;

    // for a comparison
    std::cout << "BEFORE SWAPPING" << std::endl;
    std::cout << "First number: " << fNum << std::endl;
    std::cout << "Second number: " << sNum << std::endl;
    std::cout << std::endl;

    swapNum(fNum, sNum);

    std::cout << "AFTER SWAPPING" << std::endl;
    std::cout << "First number: " << fNum << std::endl;
    std::cout << "Second number: " << sNum << std::endl;

    return 0;
}
void swapNum(int &num1, int &num2){
    int temp = num1;
    num1 = num2;
    num2 = temp;
}
```

OUTPUT


CODE ANALYSIS
This C++ program demonstrates function prototyping and pass-by-reference by swapping the values of two numbers entered by the user. At the top, void swapNum(int &num1, int &num2); is the function prototype, which tells the compiler that the swapNum() function exists and accepts two integer parameters. The & symbol means pass by reference, allowing the function to work directly with the original variables instead of making copies. This means that any changes made to num1 and num2 inside the function also change fNum and sNum in the main() function. The program first asks the user to input two numbers, displays them before swapping, then calls swapNum(fNum, sNum). Inside the function, a temporary variable temp stores the value of num1, num1 is replaced with num2, and num2 is assigned the value stored in temp, exchanging their values. After the function finishes, the updated values are displayed, showing that the swap was successful because the original variables were modified through pass-by-reference.

SECTIONS	ANSWERS
Header File Declaration Section	`#include <iostream>`
Global Declaration Section	`void swapNum(int &num1, int &num2);`

Class Declaration and Method Definition Section	**N/A**: The program does not use classes or objects.
Main Function	

```
int main(){
    int fNum, sNum;

    std::cout << "SWAP NUMBERS" << std::endl;
    std::cout << "Enter first number: ";
    std::cin >> fNum;
    std::cout << "Enter second number: ";
    std::cin >> sNum;
    std::cout << std::endl;

    // for a comparison
    std::cout << "BEFORE SWAPPING" << std::endl;
    std::cout << "First number: " << fNum << std::endl;
    std::cout << "Second number: " << sNum << std::endl;
    std::cout << std::endl;

    swapNum(fNum, sNum);

    std::cout << "AFTER SWAPPING" << std::endl;
    std::cout << "First number: " << fNum << std::endl;
    std::cout << "Second number: " << sNum << std::endl;

    return 0;
}
```

Method Definition

```
void swapNum(int &num1, int &num2){
    int temp = num1;
    num1 = num2;
    num2 = temp;
}
```

2. Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.

INPUT

```
#include <iostream>

double KelvinToFahrenheit(double kelvin);

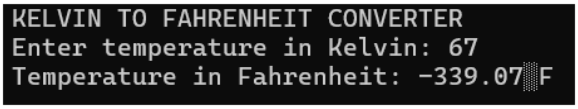
int main(){
    double kelvin, fahrenheit;

    std::cout << "KELVIN TO FAHRENHEIT CONVERTER" << std::endl;
    std::cout << "Enter temperature in Kelvin: ";
    std::cin >> kelvin;

    if (kelvin < 0){
        std::cout << "Invalid input. Kelvin temperature cannot be negative. Input new value." << std::endl;
    }
    else{
        fahrenheit = KelvinToFahrenheit(kelvin);
        std::cout << "Temperature in Fahrenheit: " << fahrenheit << "°F" << std::endl;
    }

    return 0;
}
double KelvinToFahrenheit(double kelvin){
    return (kelvin - 273.15) * 9/5 + 32;
}
```

OUTPUT





CODE ANALYSIS

This C++ program demonstrates function prototyping by converting a temperature from Kelvin to Fahrenheit using a separate function. At the top, double kelvinToFahrenheit(double kelvin); is the function prototype, which tells the compiler that the kelvinToFahrenheit() function exists, accepts one double parameter, and returns a double value. The program first asks the user to enter a temperature in Kelvin and checks whether the input is valid by ensuring it is not less than 0, since Kelvin temperatures cannot be negative. If the input is invalid, the program displays an error message. Otherwise, it calls the kelvinToFahrenheit(kelvin) function to perform the conversion. Inside the function, the formula °F = (K − 273.15) × 9/5 + 32 is used to calculate the equivalent Fahrenheit temperature, and the computed value is returned to the main() function. Finally, the program displays the converted Fahrenheit temperature, demonstrating how function prototyping and return values help organize the code by separating the conversion logic from the main program.

SECTIONS	ANSWERS
Header File Declaration Section	`#include <iostream>`
Global Declaration Section	`double KelvinToFahrenheit(double kelvin);`
Class Declaration and Method Definition Section	**N/A:** The program does not use classes or objects.

Main Function	`#include <iostream>`
	
	`double KelvinToFahrenheit(double kelvin);`
	
	`int main(){`
	`  double kelvin, fahrenheit;`
	
	`  std::cout << "KELVIN TO FAHRENHEIT CONVERTER" << std::endl;`
	`  std::cout << "Enter temperature in Kelvin: ";`
	`  std::cin >> kelvin;`
	
	`  if (kelvin < 0){`
	`  std::cout << "Invalid input. Kelvin temperature cannot be negative.`
	`  Input new value." << std::endl;`
	`  }`
	`  else{`
	`  fahrenheit = KelvinToFahrenheit(kelvin);`
	`  std::cout << "Temperature in Fahrenheit: " << fahrenheit << "°F" << std::endl;`
	`  }`
	
	`  return 0;`
	`}`
	`double KelvinToFahrenheit(double kelvin){`
	`  return (kelvin - 273.15) * 9/5 + 32;`
	`}`

Method Definition	`double KelvinToFahrenheit(double kelvin){ return (kelvin - 273.15) * 9/5 + 32; }`

3. Create a C++ program that has a function that will calculate the distance between two points.

INPUT

```
#include <iostream>
#include <cmath>

class Distance{
    private:
        double x1, y1, x2, y2;

    public:
        //constructor
        Distance(double a, double b, double c, double d);
        double calculateDistance();
};

// the constructor definition
Distance::Distance(double a, double b, double c, double d) {
    x1 = a;
    y1 = b;
    x2 = c;
    y2 = d;
}

// method definition
double Distance::calculateDistance(){
    return std::sqrt(std::pow(x2 - x1, 2) + std::pow(y2 - y1, 2));
    // formula to find the distance
}
```

```
int main(){
    double x1, y1, x2, y2;

    std::cout << "FINDING THE DISTANCE BETWEEN TWO POINTS" << std::endl;

    std::cout << "Enter x-coordinate of Point 1: ";
    std::cin >> x1;

    std::cout << "Enter y-coordinate of Point 1: ";
    std::cin >> y1;

    std::cout << "Enter x-coordinate of Point 2: ";
    std::cin >> x2;

    std::cout << "Enter y-coordinate of Point 2: ";
    std::cin >> y2;

    Distance point(x1, y1, x2, y2);

    std::cout << std::endl;
    std::cout << "DISTANCE = " << point.calculateDistance() << std::endl;

    return 0;
}
```

OUTPUT

```
FINDING THE DISTANCE BETWEEN TWO POINTS
Enter x-coordinate of Point 1: 4
Enter y-coordinate of Point 1: 5
Enter x-coordinate of Point 2: 6
Enter y-coordinate of Point 2: 7

DISTANCE = 2.82843
```

CODE ANALYSIS
This C++ program demonstrates the use of a class, constructor, and method by calculating the distance between two points entered by the user. The Distance class serves as a blueprint for creating objects that store the coordinates of two points and perform the distance calculation. It contains four private data members: x1, y1, x2, and y2, which represent the x- and y-coordinates of Point 1 (x1, y1) and Point 2 (x2, y2). In geometry, a point is an exact location on a coordinate plane and is represented by an ordered pair (x, y). The class also contains a constructor, Distance(double a, double b, double c, double d), which is a special member function that is automatically called whenever a Distance object is created. Its purpose is to initialize the object's data members by assigning the values entered by the user to x1, y1, x2, and y2. The class also declares the calculateDistance() method, which computes the distance between the two points using the Distance Formula, $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$. Inside the main() function, the user is prompted to enter the coordinates of the two points, and these values are passed to the constructor when the object point is created using Distance point(x1, y1, x2, y2);. After the object has been initialized, the program calls point.calculateDistance(), which calculates and returns the distance between the two points. The returned value is then displayed to the user. This program demonstrates how a class groups related data and functions into a single unit, while the constructor ensures that the object's data is properly initialized before its methods are used.

SECTIONS	ANSWERS
Header File Declaration Section	`#include <iostream>` `#include <cmath>`
Global Declaration Section	**N/A:** There is no global declaration used in the program.

Class Declaration and Method Definition Section	`class Distance{`
	`  private:`
	`  double x1, y1, x2, y2;`
	
	`  public:`
	`  //constructor`
	`  Distance(double a, double b, double c, double d);`
	`  double calculateDistance();`
	`};`

Main Function	
Method Definition	

```cpp
int main(){
    double x1, y1, x2, y2;

    std::cout << "FINDING THE DISTANCE BETWEEN TWO POINTS" << std::endl;

    std::cout << "Enter x-coordinate of Point 1: ";
    std::cin >> x1;

    std::cout << "Enter y-coordinate of Point 1: ";
    std::cin >> y1;

    std::cout << "Enter x-coordinate of Point 2: ";
    std::cin >> x2;

    std::cout << "Enter y-coordinate of Point 2: ";
    std::cin >> y2;

    Distance point(x1, y1, x2, y2);

    std::cout << std::endl;
    std::cout << "DISTANCE = " << point.calculateDistance() << std::endl;

    return 0;
```

```cpp
// the constructor definition
Distance::Distance(double a, double b, double c, double d) {
    x1 = a;
    y1 = b;
    x2 = c;
    y2 = d;
}

// method definition
double Distance::calculateDistance(){
    return std::sqrt((x2 - x1) * (x2 - x1) + (y2 - y1) * (y2 - y1));
    // formula to find the distance
}
```

4. Modify the code given in ILO B and add the following functions:
    a. A function to compute for the area of a triangle
    b. A function to compute for the perimeter of a triangle
    c. A function that determines whether the triangle is acute-angled, obtuse-angled or 'others.'

INPUT

```
#include <iostream>
#include <cmath>

// naming convention for class name: PascalCase
class Triangle{
    private:
        double totalAngle, angleA, angleB, angleC;
        double sideA, sideB, sideC;

    public:
        //constructor
        Triangle(double A, double B, double C, double a);

        void setAngles(double A, double B, double C, double a);
        const bool validateTriangle();

        //a. computes area
        double computeArea();

        //b. computes perimeter of a triangle
        double computePerimeter();

        //c. determine the type of a triangle
        void triangleType();
};
```

```
int main(){

    double A, B, C, a;

    std::cout << "Enter Angle A: ";
    std::cin >> A;
    std::cout << "Enter Angle B: ";
    std::cin >> B;
    std::cout << "Enter Angle C: ";
    std::cin >> C;
    std::cout << "Enter Side A: ";
    std::cin >> a;
    // using the law of Sines

    Triangle set1(A, B, C, a);

    if(set1.validateTriangle()){

        std::cout << "\nThe shape is a valid triangle.\n";

        //4a. display the area
        std::cout << "Area: " << set1.computeArea() << std::endl;

        //4b. display the perimeter
        std::cout << "Perimeter: " << set1.computePerimeter() << std::endl;

        //4c. display the type of triangle
        std::cout << "Triangle Type: ";
        set1.triangleType();
    }
    else{
        std::cout << "\nThe shape is NOT a valid triangle.";
    }

    return 0;
}
```

```
//constructor
Triangle::Triangle(double A, double B, double C, double a){

    angleA = A;
    angleB = B;
    angleC = C;
    sideA = a;

    totalAngle = A + B + C;

    //compute the remaining sides if the triangle is valid
    if(validateTriangle()){

        sideB = sideA * sin(angleB * 3.14159265358979323846 / 180)
                      / sin(angleA * 3.14159265358979323846 / 180);

        sideC = sideA * sin(angleC * 3.14159265358979323846 / 180)
                      / sin(angleA * 3.14159265358979323846 / 180);
    }
}

//set angles
void Triangle::setAngles(double A, double B, double C, double a){

    angleA = A;
    angleB = B;
    angleC = C;
    sideA = a;

    totalAngle = A + B + C;
```

```
    //recompute the remaining sides if the triangle is valid
    if(validateTriangle()){

        sideB = sideA * sin(angleB * 3.14159265358979323846 / 180)
                      / sin(angleA * 3.14159265358979323846 / 180);

        sideC = sideA * sin(angleC * 3.14159265358979323846 / 180)
                      / sin(angleA * 3.14159265358979323846 / 180);
    }
}

//validate triangle
const bool Triangle::validateTriangle(){

    return(totalAngle == 180);
}

//compute area
double Triangle::computeArea(){

    return 0.5 * sideB * sideC *
           sin(angleA * 3.14159265358979323846 / 180);
}

//compute perimeter
double Triangle::computePerimeter(){

    return sideA + sideB + sideC;
}
```

```cpp
//c. function that determine the type of a triangle
void Triangle::triangleType(){

    //acute triangle
    if(angleA < 90 && angleB < 90 && angleC < 90){
        std::cout << "Acute-angled Triangle";
    }
    //obtuse triangle
    else if(angleA > 90 || angleB > 90 || angleC > 90){
        std::cout << "Obtuse-angled Triangle";
    }
    //others
    else{
        std::cout << "Others";
    }
}
```

OUTPUT

```
Enter Angle A: 60
Enter Angle B: 60
Enter Angle C: 60
Enter Side A: 10

The shape is a valid triangle.
Area: 43.3013
Perimeter: 30
Triangle Type: Acute-angled Triangle
```

```
Enter Angle A: 30
Enter Angle B: 60
Enter Angle C: 90
Enter Side A: 5

The shape is a valid triangle.
Area: 21.6506
Perimeter: 23.6603
Triangle Type: Others
```

```
Enter Angle A: 40
Enter Angle B: 100
Enter Angle C: 40
Enter Side A: 8

The shape is a valid triangle.
Area: 31.5138
Perimeter: 28.2567
Triangle Type: Obtuse-angled Triangle
```

```
Enter Angle A: 67
Enter Angle B: 67
Enter Angle C: 67
Enter Side A: 6

The shape is NOT a valid triangle.
```

CODE ANALYSIS
This C++ program demonstrates the use of a class, constructor, and methods by calculating the area, perimeter, and type of a triangle using the three angles and one side entered by the user. The Triangle class serves as a blueprint for creating objects that store the triangle's information, including the three angles (angleA, angleB, angleC), the three sides (sideA, sideB, sideC), and the total of the three angles (totalAngle). The class contains a constructor, Triangle(double A, double B, double C, double a), which is automatically called whenever a Triangle object is created. Its purpose is to initialize the object's data members by assigning the user-inputted values to the corresponding variables. If the triangle is valid, the constructor uses the Law of Sines to calculate the remaining two sides. The Law of Sines states that the ratio of a side to the sine of its opposite angle is always equal for all three sides of a triangle. Since the program already knows Side A and Angle A, it calculates the unknown sides using the equations Side B = (Side A x sin Angle B)/ sin Angle A and Side C = (Side A x sin Angle C)/sin Angle A. Because the sin() function from the <cmath> library accepts angles in radians, the program first converts the input angles from degrees to radians before performing the calculations. The class also contains the validateTriangle() method, which checks whether the sum of the three angles is equal to 180°, ensuring that the given measurements form a valid triangle. The computeArea() method calculates the area using the equation Area = 1/2 x Side B x Side C x sin Angle A, while the computePerimeter() method computes the perimeter using Perimeter = Side A + Side B + Side C. The triangleType() method determines whether the triangle is acute-angled, obtuse-angled, or right-angled by comparing the values of its angles using conditional statements. Inside the main() function, the user is prompted to enter the three angles and one side, and these values are passed to the constructor when the Triangle object is created. The program then validates the triangle before calling the methods to calculate and display its area, perimeter, and type. This program demonstrates how classes, constructors, methods, conditional statements, mathematical formulas, and trigonometric functions from the <cmath> library can be used together to solve triangle-related calculations.

SECTIONS	ANSWERS
Header File Declaration Section	`#include <iostream>` `#include <cmath>`
Global Declaration Section	**N/A:** There is no global declaration in the program.

Class Declaration and Method Definition Section

```
// naming convention for class name: PascalCase
class Triangle{
    private:
        double totalAngle, angleA, angleB, angleC;
        double sideA, sideB, sideC;

    public:
        //constructor
        Triangle(double A, double B, double C, double a);

        void setAngles(double A, double B, double C, double a);
        const bool validateTriangle();

        //a. computes area
        double computeArea();

        //b. computes perimeter of a triangle
        double computePerimeter();

        //c. determine the type of a triangle
        void triangleType();
};
```

Main Function

```
int main(){

    double A, B, C, a;

    std::cout << "Enter Angle A: ";
    std::cin >> A;
    std::cout << "Enter Angle B: ";
    std::cin >> B;
    std::cout << "Enter Angle C: ";
    std::cin >> C;
    std::cout << "Enter Side A: ";
    std::cin >> a;
    // using the law of Sines

    Triangle set1(A, B, C, a);

    if(set1.validateTriangle()){

        std::cout << "\nThe shape is a valid triangle.\n";

        //4a. display the area
        std::cout << "Area: " << set1.computeArea() << std::endl;

        //4b. display the perimeter
        std::cout << "Perimeter: " << set1.computePerimeter() << std::endl;

        //4c. display the type of triangle
        std::cout << "Triangle Type: ";
        set1.triangleType();
    }
    else{
        std::cout << "\nThe shape is NOT a valid triangle.";
    }

    return 0;
}
```

Method Definition

```
//constructor
Triangle::Triangle(double A, double B, double C, double a){

    angleA = A;
    angleB = B;
    angleC = C;
    sideA = a;

    totalAngle = A + B + C;

    //compute the remaining sides if the triangle is valid
    if(validateTriangle()){

        sideB = sideA * sin(angleB * 3.14159265358979323846 / 180)
                      / sin(angleA * 3.14159265358979323846 / 180);

        sideC = sideA * sin(angleC * 3.14159265358979323846 / 180)
                      / sin(angleA * 3.14159265358979323846 / 180);
    }
}

//set angles
void Triangle::setAngles(double A, double B, double C, double a){

    angleA = A;
    angleB = B;
    angleC = C;
    sideA = a;

    totalAngle = A + B + C;


    //recompute the remaining sides if the triangle is valid
    if(validateTriangle()){

        sideB = sideA * sin(angleB * 3.14159265358979323846 / 180)
                      / sin(angleA * 3.14159265358979323846 / 180);

        sideC = sideA * sin(angleC * 3.14159265358979323846 / 180)
                      / sin(angleA * 3.14159265358979323846 / 180);
    }
}

//validate triangle
const bool Triangle::validateTriangle(){

    return(totalAngle == 180);
}

//compute area
double Triangle::computeArea(){

    return 0.5 * sideB * sideC *
           sin(angleA * 3.14159265358979323846 / 180);
}

//compute perimeter
double Triangle::computePerimeter(){

    return sideA + sideB + sideC;
}
```

```cpp
//c. function that determine the type of a triangle
void Triangle::triangleType(){

    //acute triangle
    if(angleA < 90 && angleB < 90 && angleC < 90){
        std::cout << "Acute-angled Triangle";
    }
    //obtuse triangle
    else if(angleA > 90 || angleB > 90 || angleC > 90){
        std::cout << "Obtuse-angled Triangle";
    }
    //others
    else{
        std::cout << "Others";
    }
}
```

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT, Grammarly

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Grammar correction, idea generation, and code assistance.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: AI was used as a support tool only. Grammarly was used for grammar correction in the code analysis, while ChatGPT was used for idea generation, improving implementation approaches, and assisting with debugging. AI did not replace my original thinking, analysis, authorship, or completion of the required work. All coding, implementation, testing, and final outputs were completed by me.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.